WEEK 4: NOSQL THINKING + MONGODB

SQL vs NoSQL - Fundamental Differences

SQL (Relational):

- Structured schema
- Tables + relationships
- ACID transactions
- Query language: SQL
- Examples: PostgreSQL, MySQL, SQLite

NoSQL:

- Flexible schema
- Collections + documents
- Consistency model depends on the database and read/write configuration
- Query language: JSON-like (MongoDB), custom, etc.
- Examples: MongoDB, Cassandra, DynamoDB, Redis

Key difference:

- SQL: "Schema first, then data"
- NoSQL: "Data first, schema flexible"

When to Use NoSQL

Use NoSQL when:

- Schema is uncertain or evolving
- Data is unstructured (logs, JSON)
- Massive scale (millions of writes/sec)
- Horizontal scaling needed
- No complex joins required
- Flexibility > ACID guarantees

Use SQL when:

- Strong schema requirement
- Complex relationships (many JOINs)
- ACID transactions critical
- Data integrity essential
- Querying ad-hoc patterns

This week: MongoDB teaches NoSQL thinking.

Document Model vs Relational Model

Relational (SQL):

```
CUSTOMERS            ORDERS                  ITEMS
+----+-------+       +----+----+------+      +----+-----+-------+
| id | name  |       | id | cid|amount|      | id | oid | item  |
+----+-------+       +----+----+------+      +----+-----+-------+
```

Multiple normalized tables, JOINs to reconstruct.

Document (NoSQL):

```
{
  "_id": 1, "name": "Alice",
  "orders": [{
    "order_id": 101, "amount": 100,
    "items": [
      {"name": "Shoe", "qty": 2},
      {"name": "Shirt", "qty": 1}
    ]
  }]
}
```

Single document, nested structure.

Embedding vs Referencing

Embedding: Nest related data inside a document.

When to embed:

- Data always accessed together
- Moderate data size
- Few/no updates to nested data
- One-to-few relationship

Embedding example:

```
{
  "_id": 1,
  "name": "Alice",
  "address": {
    "street": "123 Main",
    "city": "NYC"
  },
  "phone_numbers": ["555-1234", "555-5678"]
}
```

Referencing: Store document IDs, link separately.

When to reference:

- Data accessed independently
- Large/changing nested data
- Many-to-many relationships
- Multiple documents reference same data

Referencing example:

```
CUSTOMERS:
{ "_id": 1, "name": "Alice", "address_id": 100 }

ADDRESSES:
{ "_id": 100, "street": "123 Main", "city": "NYC" }
```

MongoDB Basics

MongoDB = Document database storing JSON-like documents.

Key concepts:

- Database: Container for collections
- Collection: Container for documents
- Document: JSON-like object with fields
- Field: Key-value pair

Example document (from MongoDB official inventory collection):

```
{
  "_id": ObjectId("6a1989e89ee967ac248ce5b0"),
  "item": "canvas",
  "qty": 100,
  "status": "A",
  "tags": ["cotton"],
  "dim_cm": [28, 35.5],
  "size": { "h": 28, "w": 35.5, "uom": "cm" }
}
```

Key features:

- `_id` : Auto-generated unique identifier
- `item` , `qty` , `status` : Simple fields
- `tags` : Array field
- `dim_cm` : Numeric array
- `size` : Embedded document (nested object)

How to use MongoDB

MongoDB in Docker (with authentication):

```
docker run -d \
  -p 27017:27017 \
  -e MONGODB_INITDB_ROOT_USERNAME=mongo \
  -e MONGODB_INITDB_ROOT_PASSWORD=mongo \
  --name mongodb83 \
  mongo:latest
```

MongoDB Schema Design Process (Official Workflow)

MongoDB documentation recommends this sequence:

1. Identify workload (most frequent reads/writes)
2. Map relationships (1:1, 1:N, N:N)
3. Apply design patterns (embed, reference, duplicate selectively)
4. Create indexes to support query patterns

Core rule: Data accessed together should be stored together.

Practical decision checklist:

- Embed if reads need one document and children are bounded.
- Reference if child cardinality is high or queried independently.
- Embed to get single-document atomic updates.
- Reference to reduce duplication and update fan-out.

Data Modelling Decision Example

Access pattern: Product page needs product + latest 5 reviews in one request.

Before (fully referenced):

```
PRODUCTS: { "_id": "p1", "name": "Laptop" }
REVIEWS:  { "_id": "r1", "product_id": "p1", "rating": 5, "text": "Great" }
```

After (hybrid model):

```
{
  "_id": "p1", "name": "Laptop",
  "latest_reviews": [
    { "rating": 5, "text": "Great", "at": "2026-01-01" },
    { "rating": 4, "text": "Good",  "at": "2026-01-02" }
  ]
}
```

REVIEWS full history collection remains separate.

Why: Fast reads with embedded summary; full history still queryable.

CRUD Operations - Create

Insert one document (official pattern):

```
db.inventory.insertOne({
  item: "canvas",
  qty: 100,
  tags: ["cotton"],
  size: { h: 28, w: 35.5, uom: "cm" }
})
```

Insert multiple documents (official pattern):

```
db.inventory.insertMany([
  { item: "journal",  qty: 25,  status: "A", size: { h: 14,  w: 21, uom: "cm" } },
  { item: "notebook", qty: 50,  status: "A", size: { h: 8.5, w: 11, uom: "in" } },
  { item: "paper",    qty: 100, status: "D", size: { h: 8.5, w: 11, uom: "in" } }
])
```

Auto-generated _id:

```
Each document gets a unique _id (like primary key)
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "Alice",
  ...
}
```

Before insert:

```
db.inventory.find({ item: "canvas" }).toArray()
```

```
[]
```

Insert operation:

```
db.inventory.insertOne({
  item: "canvas", qty: 100, status: "A",
  tags: ["cotton"], dim_cm: [28, 35.5],
  size: { h: 28, w: 35.5, uom: "cm" }
})
```

After insert:

```
db.inventory.find({ item: "canvas" }).toArray()
```

```
[{
  _id: ObjectId('6a1989e89ee967ac248ce5b0'),
  item: 'canvas', qty: 100, status: 'A',
  tags: ['cotton'], dim_cm: [28, 35.5],
  size: { h: 28, w: 35.5, uom: 'cm' }
}]
```

Notice: MongoDB auto-generates the `_id` field if not provided.

Collection after all CREATE steps:

```
db.inventory.find({}, { _id: 0, item: 1, qty: 1, status: 1, "size.uom": 1 }).toArray()
```

```
[
  { item: 'canvas',   qty: 100, status: 'A', size: { uom: 'cm' } },
  { item: 'journal',  qty: 25,  status: 'A', size: { uom: 'cm' } },
  { item: 'notebook', qty: 50,  status: 'A', size: { uom: 'in' } },
  { item: 'paper',    qty: 100, status: 'D', size: { uom: 'in' } },
  { item: 'planner',  qty: 75,  status: 'D', size: { uom: 'cm' } },
  { item: 'postcard', qty: 45,  status: 'A', size: { uom: 'cm' } }
]
```

CRUD Operations - Read

Find all documents:

```
db.inventory.find({})
```

Find with filter (WHERE equivalent):

```
db.inventory.find({ status: "D" })
db.inventory.find({ status: { $in: ["A", "D"] } })
```

Common operators:

```
$eq    = equal           $ne   = not equal
$gt    = greater than    $gte  = greater or equal
$lt    = less than       $lte  = less or equal
$in    = in array        $nin  = not in array
```

Project specific fields (SELECT equivalent):

```
db.inventory.find(
  { status: "A" },
  { item: 1, status: 1, _id: 0 }
)
```

Nested document queries (dot notation):

```
db.inventory.find({ "size.uom": "in" })
db.inventory.find({ "size.h": { $lt: 15 }, "size.uom": "in", status: "D" })
```

Array queries:

```
db.inventory.find({ tags: "red" })
db.inventory.find({ tags: { $all: ["red", "blank"] } })
db.inventory.find({ dim_cm: { $elemMatch: { $gt: 22, $lt: 30 } } })
```

Full document query:

```
db.inventory.find({ status: "A" }).toArray()
```

```
[
  {
    _id: ObjectId('6a1989e89ee967ac248ce5b0'),
    item: 'canvas', qty: 100, status: 'A',
    tags: ['cotton'], dim_cm: [28, 35.5],
    size: { h: 28, w: 35.5, uom: 'cm' }
  }, ...
]
```

Projected query (fewer fields):

```
db.inventory.find({ status: "A" }, { item: 1, status: 1, _id: 0 }).toArray()
```

```
[
  { item: 'canvas',   status: 'A' },
  { item: 'journal',  status: 'A' },
  { item: 'notebook', status: 'A' },
  { item: 'postcard', status: 'A' }
]
```

Key insight: Projection reduces document size and network transfer.

Actual read result - nested field query:

```
db.inventory.find({ "size.uom": "in" }, { _id: 0, item: 1, status: 1, "size.uom": 1 }).toArray()
```

```
[
  { item: 'notebook', status: 'A', size: { uom: 'in' } },
  { item: 'paper',    status: 'D', size: { uom: 'in' } }
]
```

Actual read result - array query:

```
db.inventory.find({ tags: { $all: ["red", "blank"] } }, { _id: 0, item: 1, tags: 1 }).toArray()
```

```
[
  { item: 'journal',  tags: ['blank', 'red'] },
  { item: 'notebook', tags: ['red', 'blank'] },
  { item: 'paper',    tags: ['red', 'blank', 'plain'] },
  { item: 'planner',  tags: ['blank', 'red'] }
]
```

CRUD Operations - Update

Update one document (official pattern):

```
db.inventory.updateOne(
  { item: "paper" },
  {
    $set: { "size.uom": "cm", status: "P" },
    $currentDate: { lastModified: true }
  }
)
```

Update multiple documents (official pattern):

```
db.inventory.updateMany(
  { qty: { $lt: 50 } },
  {
    $set: { "size.uom": "in", status: "P" },
    $currentDate: { lastModified: true }
  }
)
```

Update operators:

```
$set         = set field value
$unset       = remove field
$inc         = increment value
$push        = add to array
$pull        = remove from array
$currentDate = set field to current date/time
```

Example: Increment quantity

```
db.inventory.updateOne(
  { item: "journal" },
  { $inc: { qty: 5 } }
)
```

Before update:

```
db.inventory.find({ item: "paper" }).toArray()
```

```
[{
  _id: ObjectId('6a1989e89ee967ac248ce5b3'),
  item: 'paper', qty: 100, status: 'D',
  tags: ['red', 'blank', 'plain'],
  dim_cm: [14, 21], size: { h: 8.5, w: 11, uom: 'in' }
}]
```

Update operation:

```
db.inventory.updateOne(
  { item: "paper" },
  {
    $set: { "size.uom": "cm", status: "P" },
    $currentDate: { lastModified: true }
  }
)
```

After update:

```
db.inventory.find({ item: "paper" }).toArray()
```

```
[{
  _id: ObjectId('6a1989e89ee967ac248ce5b3'),
  item: 'paper', qty: 100, status: 'P',
  tags: ['red', 'blank', 'plain'], dim_cm: [14, 21],
  size: { h: 8.5, w: 11, uom: 'cm' },
  lastModified: ISODate('2026-05-29T12:43:20.398Z')
}]
```

Key changes: `status` D→P, `size.uom` in→cm, `lastModified` added.

Before array update:

```
db.inventory.find({ item: "journal" }, { _id: 0, item: 1, tags: 1 }).toArray()
```

```
[{ item: 'journal', tags: ['blank', 'red'] }]
```

Array update operation:

```
db.inventory.updateOne(
  { item: "journal" },
  { $push: { tags: "featured" } }
)
```

Array operators:

- `$push` : Add element to end of array
- `$pull` : Remove element from array
- `$addToSet` : Add only if not already present

After array update:

```
db.inventory.find({ item: "journal" }, { _id: 0, item: 1, tags: 1 }).toArray()
```

```
[{ item: 'journal', tags: ['blank', 'red', 'featured'] }]
```

Collection after all UPDATE steps:

```
db.inventory.find({}, { _id: 0, item: 1, qty: 1, status: 1, tags: 1, "size.uom": 1 }).toArray()
```

```
[
  { item: 'canvas',   qty: 100, status: 'A', tags: ['cotton'],                 size: { uom: 'cm' } },
  { item: 'journal',  qty: 30,  status: 'P', tags: ['blank', 'red', 'featured'], size: { uom: 'in' } },
  { item: 'notebook', qty: 50,  status: 'A', tags: ['red', 'blank'],           size: { uom: 'in' } },
  { item: 'paper',    qty: 100, status: 'P', tags: ['red', 'blank', 'plain'],  size: { uom: 'cm' } },
  { item: 'planner',  qty: 75,  status: 'D', tags: ['blank', 'red'],           size: { uom: 'cm' } },
  { item: 'postcard', qty: 45,  status: 'P', tags: ['blue'],                   size: { uom: 'in' } }
]
```

CRUD Operations - Delete

Delete one document:

```
db.inventory.deleteOne({ status: "D" })
```

Delete multiple documents:

```
db.inventory.deleteMany({ status: "A" })
```

Delete all documents (careful!):

```
db.inventory.deleteMany({})
```

Before delete:

```
db.inventory.find({ status: "D" }).count()
```

```
1
```

```
db.inventory.find({ status: "A" }).count()
```

```
2
```

Delete operations:

```
db.inventory.deleteOne({ status: "D" })
db.inventory.deleteMany({ status: "A" })
```

After delete:

```
db.inventory.find({}).toArray()
```

```
[
  { _id: ObjectId('...'), item: 'journal',  qty: 30,  status: 'P', ... },
  { _id: ObjectId('...'), item: 'paper',    qty: 100, status: 'P', ... },
  { _id: ObjectId('...'), item: 'postcard', qty: 45,  status: 'P', ... }
]
```

Only 3 documents remain (all with status 'P').

Aggregation Pipeline

The **aggregation pipeline** is MongoDB's powerful multi-stage processing.

Think of it like:

1. Filter data (`$match`)
2. Transform shape (`$project`)
3. Group and aggregate (`$group`)
4. Sort (`$sort`) / Limit (`$limit`)

Syntax:

```
db.collection.aggregate([
  { $stage1: {...} },
  { $stage2: {...} },
  { $stage3: {...} }
])
```

Data flows through each stage like a pipe.

Aggregation - $match and $project

$match: Filter documents (WHERE equivalent)

```
db.inventory.aggregate([
  { $match: { qty: { $gt: 50 } } }
])
```

$project: Select/transform fields (SELECT equivalent)

```
db.inventory.aggregate([
  { $project: { item: 1, inventory_status: "$status", unit: "$size.uom" } }
])
```

Aggregation - $group

$group: Aggregate data (GROUP BY equivalent)

```
db.inventory.aggregate([
  { $group: {
      _id: "$status",
      total_qty: { $sum: "$qty" },
      num_items: { $sum: 1 },
      avg_qty:   { $avg: "$qty" }
    }
  }
])
```

Result:

```
[
  { _id: 'P', total_qty: 175, num_items: 3, avg_qty: 58.33 },
  { _id: 'A', total_qty: 150, num_items: 2, avg_qty: 75 },
  { _id: 'D', total_qty: 75,  num_items: 1, avg_qty: 75 }
]
```

Totals reflect collection state after the update steps in the script.

Before $group:

```
{ item: 'canvas',   qty: 100, status: 'A' }
{ item: 'journal',  qty: 30,  status: 'P' }
{ item: 'notebook', qty: 50,  status: 'A' }
{ item: 'paper',    qty: 100, status: 'P' }
{ item: 'planner',  qty: 75,  status: 'D' }
{ item: 'postcard', qty: 45,  status: 'P' }
```

After $group:

```
{ _id: 'P', total_qty: 175 }  { _id: 'A', total_qty: 150 }  { _id: 'D', total_qty: 75 }
```

6 documents → 3 aggregated summaries.

Aggregation operators:

```
$sum   = sum values          $avg   = average
$min   = minimum             $max   = maximum
$count = count documents     $first = first value
$last  = last value          $push  = collect into array
```

Aggregation - $cond for Conditional Logic

$cond: If-then-else in aggregation

```
db.inventory.aggregate([
  { $group: {
      _id: "$status",
      high_qty_docs: { $sum: { $cond: [{ $gt: ["$qty", 50] }, 1, 0] } },
      total_qty: { $sum: "$qty" }
    }
  }
])
```

Result:

```
[
  { _id: 'A', high_qty_docs: 1, total_qty: 150 },
  { _id: 'D', high_qty_docs: 1, total_qty: 75  },
  { _id: 'P', high_qty_docs: 1, total_qty: 175 }
]
```

- Status 'A': canvas (qty 100) > 50
- Status 'D': planner (qty 75) > 50
- Status 'P': paper (qty 100) > 50

Document Modelling Patterns

1. Embedded 1:1:

```
{ "_id": 1, "name": "Alice",
  "address": { "street": "123 Main", "city": "NYC" } }
```

2. Array of embedded 1:N:

```
{ "_id": 1, "name": "Alice", "orders": [
  { "order_id": 101, "amount": 100 },
  { "order_id": 102, "amount": 200 }
] }
```

3. Referenced N:N:

```
ORDERS:   { "_id": 101, "customer_id": 1, "product_ids": [1, 2, 3] }
PRODUCTS: { "_id": 1,   "name": "Shoe" }
```

Trade-off:

- Embedding: Faster reads, larger documents
- Referencing: Flexibility, smaller documents

MongoDB constraints:

- Document size limit is 16 MiB.
- Avoid unbounded array growth in embedded models.
- Use references for high-cardinality or independently queried child data.

Atomicity and Modelling

MongoDB write operations are atomic at single-document level.

Before (split documents):

```
BOOK:      { "_id": 1, "available": 3 }
CHECKOUTS: { "book_id": 1, "by": "alice" }
```

Needs multiple writes to stay consistent.

After (embedded checkout list):

```
{
  "_id": 1, "title": "MongoDB Guide", "available": 2,
  "checkout": [
    { "by": "alice", "date": "2026-01-15" }
  ]
}
```

Can update `available` and `checkout` in one atomic update.

NoSQL vs SQL - Thinking Shift

SQL thinking:

- Normalize everything
- Avoid repetition
- JOIN tables for queries
- Schema design first

NoSQL thinking:

- Denormalize for access patterns
- Accept repetition
- Embed related data
- Query patterns first

SQL (normalized):

```
CUSTOMERS -> ORDERS -> ORDER_ITEMS -> PRODUCTS
(4 tables, JOINs required)
```

NoSQL (denormalized):

```
Customer document with embedded orders
(1 document, no JOINs)
```

Choose the model that fits your queries.

Summary

Week 4 covered:

- SQL vs NoSQL fundamentals
- Document model and flexibility
- MongoDB schema design workflow
- Embedding vs referencing strategies
- MongoDB CRUD operations
- Aggregation pipeline (match, project, group, cond)
- Document modelling patterns and NoSQL thinking

You can now:

- Design MongoDB collections
- Write aggregation pipelines
- Choose embedding vs referencing
- Think about flexible schemas

Run classroom script:

```
docker exec -i mongodb83 mongosh -u mongo -p mongo \
  --authenticationDatabase admin --quiet \
  --eval "$(cat code/week4.js)"
```

Next week: Graph databases, Recursive SQL, Apache AGE, Knowledge graphs.

EXERCISES

Exercise 1: MongoDB Setup and Basic CRUD

Start MongoDB in Docker:

```
docker run -d \
  -p 27017:27017 \
  -e MONGODB_INITDB_ROOT_USERNAME=mongo \
  -e MONGODB_INITDB_ROOT_PASSWORD=mongo \
  --name mongodb83 \
  mongo:latest
```

Connect and create a database:

```
docker exec -it mongodb83 mongosh -u mongo -p mongo --authenticationDatabase admin
```

```
use ecommerce
```

Create customers collection:

```
db.customers.insertMany([
  { "_id": 1, "name": "Alice",   "email": "alice@example.com",   "age": 30, "segment": "premium"  },
  { "_id": 2, "name": "Bob",     "email": "bob@example.com",     "age": 25, "segment": "standard" },
  { "_id": 3, "name": "Charlie", "email": "charlie@example.com", "age": 35, "segment": "premium"  }
])
```

Queries:

```
// 1. Find all customers
db.customers.find({}).toArray()
// 2. Find customers with age > 30
db.customers.find({ age: { $gt: 30 } }).toArray()
// 3. Update Alice's segment to "vip"
db.customers.updateOne({ name: "Alice" }, { $set: { segment: "vip" } })
// 4. Delete Bob
db.customers.deleteOne({ name: "Bob" })
// 5. Find remaining customers
db.customers.find({}).toArray()
```

Expected result: Alice (vip), Charlie (premium) remain; Bob is removed.

Exercise 2: Document Design - Embedded Data

Create orders collection:

```
db.orders.insertMany([
  { "_id": 101, "customer_id": 1, "order_date": "2024-01-15", "total": 1059.97,
    "items": [
      { "product": "Laptop", "qty": 1, "price": 999.99 },
      { "product": "Mouse",  "qty": 2, "price": 29.99  }] },
  { "_id": 102, "customer_id": 1, "order_date": "2024-01-20", "total": 129.99,
    "items": [{ "product": "Keyboard", "qty": 1, "price": 129.99 }] },
  { "_id": 103, "customer_id": 2, "order_date": "2024-01-18", "total": 399.99,
    "items": [{ "product": "Monitor", "qty": 1, "price": 399.99 }] }
])
```

Queries:

1. Find all orders
2. Find orders with total > 500
3. Find orders containing "Laptop"
4. Update order 101 to add "Speaker" with price 79.99
5. Find customer 1's orders

Exercise 3: Aggregation Pipeline - Basic

Total spending by customer:

```
db.orders.aggregate([
  { $group: {
      _id: "$customer_id",
      total_spent: { $sum: "$total" },
      num_orders:  { $sum: 1 },
      avg_order:   { $avg: "$total" }
    }
  },
  { $sort: { total_spent: -1 } }
])

---

**Expected result:**
```

Customer 1: spent $1189.96 in 2 orders (avg $594.98)

Find high-value items (price > 100):

```
db.orders.aggregate([
  { $unwind: "$items" },
  { $match:  { "items.price": { $gt: 100 } } },
  { $project: { "items.product": 1, "items.price": 1 } }
])
```

Exercise 4: Aggregation with $cond

Count high-value vs low-value orders per customer:

```
db.orders.aggregate([
  { $group: {
      _id: "$customer_id",
      total_orders:      { $sum: 1 },
      high_value_orders: { $sum: { $cond: [{ $gt:  ["$total", 500] }, 1, 0] } },
      low_value_orders:  { $sum: { $cond: [{ $lte: ["$total", 500] }, 1, 0] } },
      total_spent:       { $sum: "$total" }
    }
  }
])
```

Expected result:

```
Customer 1: 2 orders (1 high-value >$500, 1 low-value), spent $1189.96
Customer 2: 1 order  (0 high-value, 1 low-value),       spent $399.99
```

Exercise 5: Design a New Collection

Create a products collection:

```
db.products.insertMany([
  { "_id": "laptop", "name": "Laptop", "price": 999.99, "category": "Electronics" },
  { "_id": "mouse",  "name": "Mouse",  "price": 29.99,  "category": "Electronics" }
])
```

Modify orders to reference products by ID:

```
db.orders.updateOne(
  { "_id": 101 },
  { $set: {
      items: [
        { product_id: "laptop", qty: 1, price: 999.99 },
        { product_id: "mouse",  qty: 2, price: 29.99  }
      ]
    }
  }
)
```

Query orders with product details ($lookup):

```
db.orders.aggregate([
  {
    $lookup: {
      from:         "products",
      localField:   "items.product_id",
      foreignField: "_id",
      as:           "product_details"
    }
  }
])
```

Exercise 6: Reflection

1. When would you embed vs reference?
   (Hint: Think about access patterns and data size)
2. What's the difference between SQL JOIN and MongoDB `$lookup`?
3. In the orders collection, is the embedded structure best or should items be referenced?
4. Design a MongoDB collection for a blog:
   - Posts with embedded comments
   - What fields would each have?
   - Would you embed or reference the author?